CLRS • 第 16 章 摊还分析 • 习题 16-3 （原 17-3 a–e）

摊还加权平衡树

重建、势能，与 O(lg n) 的摊还界

① 不旋转：失衡就把子树重建成 1/2- 平衡

② 势函数 Φ = c·Σ{Δ≥2} Δ 替这次重建付账

③ 195 行 Python + 15 组实测

证明 + 源码 + 实测

a–e 五问逐条作答

code/wbt.py · code/exp.py

2026.10.18 · α 为平衡参数（1/2 ≤ α < 1） · 全部数字来自 code/data.json

导览

三句话看懂，然后按路线走一遍

三句话

① size 域

每个结点记住子树大小；插入 / 删除只沿路径 O(lg n) 更新。

② 最高失衡点

某侧规模 > α·size 就叫失衡；只重建离根最近的那个失衡点，一次修好整条路径。

③ 势能付账

Φ = c·Σ{Δ≥2} Δ：树越歪 Φ 越大，重建时它跌下来，正好抵掉重建的代价。

④ 代价

单次最坏 O(n)（恰好在根处重建），摊还 O(lg n)，两句话都对。

⑤ α 怎么选

实测 0.75–0.85 最优：太小则重建频繁，太大则树变深。

路线

1. 题面与记号： α- 平衡、 Δ 、 Φ
2. (a) 中序 + 二分重建： Θ(m)
3. (b) 每层丢（1−α)： 搜索 O(lg n)
4. (c)(d) 非负、 1/2- 平衡为 0 、 c ≥ 1/(2α−1)
5. (e) 插入 / 删除的摊还账本
6. 源码： insert / delete 怎么找失衡点
7. 算法特点与实测（ 2 张图）
8. a–e 的一句话答案

16-3

16-3 题面： α- 平衡 + 重建，五个小问

定义（题面）

size(x) 以 x 为根的子树中关键字个数

α- 平衡 left.size ≤ α·size 且 right.size ≤ α·size， 1/2 ≤ α < 1

Δ(x) |x.left.size − x.right.size|

Φ(T) c·Σ{Δ(x) ≥ 2} Δ(x)， c 依赖 α

为什么 α 不能小于 1/2

左右之和 = size − 1 。若 α < 1/2，则两侧都 ≤ α·size 时 size − 1 ≤ 2α·size < size，矛盾 —— 故题设 α ≥ 1/2 。

五个小问

a. 任意 BST 中结点 x，如何重建其子树成 1/2- 平衡，Θ(x.size) 、 O(x.size) 空间

b. n 结点 α- 平衡树中一次搜索最坏 O(lg n)

c. 任意 BST 的势非负； 1/2- 平衡树的势为 0

d. 若 m 单位势能可支付重建 m 结点子树， c 应多大才使重建摊还 O(1)

e. n 结点 α- 平衡树中插入 / 删除的摊还时间为 O(lg n)

题眼：**重建只能发生在「最高的」失衡点。**它一次性把整条路径上的失衡全部修好，同时让被重建子树里所有结点的 Δ 归零 —— 这正是势能能「付账」的地方。

16-3

16-3(a)　中序 + 二分 = 1/2- 平衡，且是 Θ(m)

```python
def inorder(t):                # ① 迭代中序 Θ(m)
    out, st = [], []
    while st or t:
        while t:
            st.append(t); t = t.l
        t = st.pop(); out.append(t.k); t = t.r
    return out

def build(a, lo, hi):          # ② 中位数递归 Θ(m)
    if lo > hi: return None
    mid = (lo + hi) // 2
    t = Node(a[mid])
    t.l = build(a, lo, mid - 1)
    t.r = build(a, mid + 1, hi)
    return pull(t)             # pull: sz = 1 + 左 + 右

def rebuild_inplace(t):        # ③ 原地重建，父指针不动
    a = inorder(t)
    new = build(a, 0, len(a) - 1)
    t.k, t.l, t.r, t.sz = new.k, new.l, new.r, new.sz
    return len(a)              # 代价 = m
```

为什么正确

- 中序序列不变 ⟹ 关键字顺序不变，仍是合法 BST
- 每层取中位数为根 ⟹ 左右规模差 ≤ 1 ⟹ 每点 Δ ≤ 1 ⟹ **1/2- 平衡**

为什么 Θ(m)

- 中序：每点进出栈各一次， Θ(m)
- 建树： T(m) = 2T(m/2) + Θ(1) ⟹ **Θ(m)**
- 空间：数组 O(m) + 递归栈 O(lg m)

实现细节

- 原地改写 t 的字段，父结点仍指向 t，**不必回填指针**
- 递归深度 O(lg m)，不会爆栈

顺带一句： Θ(m) 已经最优

任何重建都必须至少看一遍全部 m 个结点（漏掉的关键字就丢了），所以 Θ(m) 时间 + O(m) 空间是不可再省的下界。

16-3(b)　每下一层至少丢掉（1−α）的结点

证明（三行）

设从根出发走 d 步到达结点 v，则 v.size ≤ α^d · n。
走到叶子时 v.size ≥ 1，故 α^d·n ≥ 1，即 d ≤ log_{1/α} n。
　d ≤ lg n / lg(1/α) = O(lg n)。■

注意 α 的位置

α 越接近 1/2，界越紧（≈ lg n）；α 越接近 1，界越松（常数 1/lg(1/α) 变大）。

几组具体常数

α = 0.5　深度 ≤ lg n
α = 0.6　≤ log_{5/3} n ≈ 1.36 lg n
α = 0.75　≤ log_{4/3} n ≈ 2.41 lg n

实测对照（α=0.75，n=2×10⁴）

理论界 log_{4/3}(2×10⁴) ≈ 34；
实测树高 20，一次操作平均访问 14.8 个结点 —— 界不紧，但阶数正确。

把 n 换成任意子树规模，结论照样成立 ——
(e) 里「路径长度 = O(lg n)」就是这么来的。

16-3

16-3(c)(d)　非负、 1/2- 平衡为零，以及 c 到底要多大

(c) 势非负，且 1/2- 平衡时为零

- Δ(x) 是绝对值 ⟹ Δ ≥ 0 ⟹ 每项非负 ⟹ **Φ ≥ 0**
- 1/2- 平衡： left + right = size − 1 且两边 ≤ size/2 ⟹ |Δ| ≤ 1
- 于是「Δ ≥ 2」的项一个也没有 ⟹ **Φ = 0**

(d) 一次重建的势能落差

设重建子树根为 x、规模 m，且（不妨） left > α·m：

right < (1−α)m ⟹ **Δ(x) > (2α−1)m**

重建后子树内全为 1/2- 平衡，这部分势降到 0。

摊还一行算式（题目的核心）

实际代价 m　+　势能变化 ΔΦ ≤ m − c·(2α−1)m

要让它 ≤ 0，只需 **c ≥ 1/(2α−1)**。取等号时「刚好自付」，再大一点更安全。

一个容易漏掉的点：重建**不改变子树大小**，所以祖先的 size、Δ 全都原封不动 ——

势能不会在树的上方偷偷涨回来，这笔账才赖得掉。

16-3(e)　一次插入 / 删除的账本：O(lg n) + 0

阶段	做了什么	代价	势能变化
① 下降	沿根到叶的路径比较、判断方向	≤ log_{1/α} n	0
② 改 size	路径上每个结点 size +1 （或 − 1 ）	O(lg n)	≤ c·log_{1/α} n
③ 找失衡点	自顶向下找「最高的」失衡结点	O(lg n)	0
④ 重建	中序 + 二分，重建该子树（至多一次）	m	≤ −c(2α−1)m
合计	摊还代价	**O(lg n)**	**O(lg n)**

为什么只有一个 O(lg n) 的势能增量：

只有路径上的结点 Δ 会变，每次最多变 1；路径长度 log_{1/α} n 。

为什么重建摊还后是 0： 由 (d)， m ≤ c(2α−1)m 当 c ≥ 1/(2α−1) 。

删除要多说一句

删除时 size 沿两类路径变化（原位置 + 后继位置），仍是 O(lg n) 个结点。

「最高的失衡点」重建一次，覆盖其下所有失衡 —— 结论不变。

16-3

16-3 源码：插入时怎么找到「最高失衡点」

```python
def insert(root, key, alpha):
    z = Node(key)
    if root is None: return z, 1
    path, t = [], root
    while True:                          # ① 普通 BST 插入
        path.append(t)
        if key < t.k:
            if t.l is None: t.l = z; break
            t = t.l
        else:
            if t.r is None: t.r = z; break
            t = t.r
    for u in reversed(path): pull(u) # ② 自底向上修 size
    cost = len(path)
    for u in path:                       # ③ 自顶向下：先撞到的就是最高
        if unbalanced(u, alpha):
            cost += rebuild_inplace(u)   # ④ 只重建一次
            break
    return root, cost
```

三个设计选择

- **path 数组**代替父指针：省一个字段，也省了回填
- **自底向上** pull：先有孩子的 size，才有父亲的
- **自顶向下** 找失衡：第一个命中的就是最高的

```python
# delete：同样两段循环
for u in reversed(st["path"]):
    pull(u)                          # 先修 size
for u in reversed(st["path"]):       # 自根向下
    if unbalanced(u, alpha):
        rebuild_inplace(u); break
```

删除的两孩子情形用后继顶替，路径会多出一段，
但仍是 O(lg n) 个结点，结论不变。

16-3

16-3 算法特点：用「一次性重建」换掉所有旋转

维度	加权平衡树（本题）	AVL / 红黑树（对照）
单次最坏	O(n)：可能重建整棵子树	O(lg n)
摊还	**O(lg n)**	O(1) 旋转
额外空间	O(m) 临时数组（重建时）	O(1)
维护动作	**只有重建，没有旋转**	旋转 + 染色
实现难度	低：中序 + 二分就完了	高：四种旋转 / 染色 case
常数	写放大（要复制整棵子树）	局部修改，缓存友好

一句话

它把「平衡」从「每次都维持」改成了「偶尔大修一次」—— 摊还分析最典型的形态。

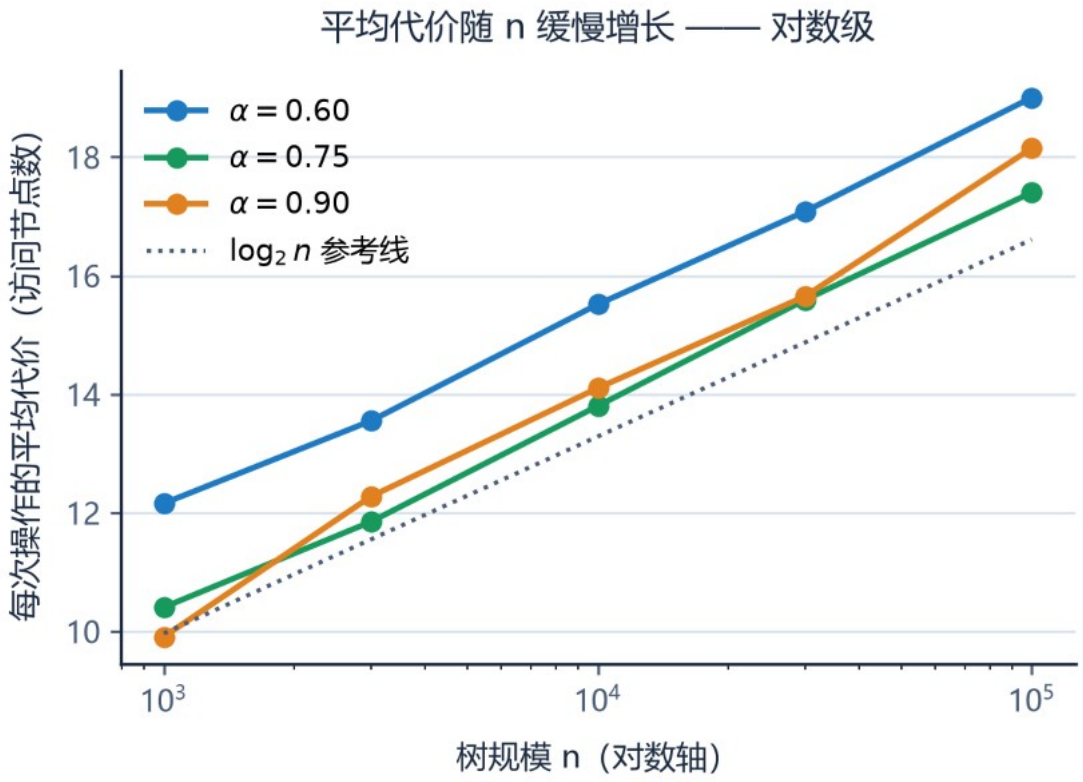
什么时候值得用

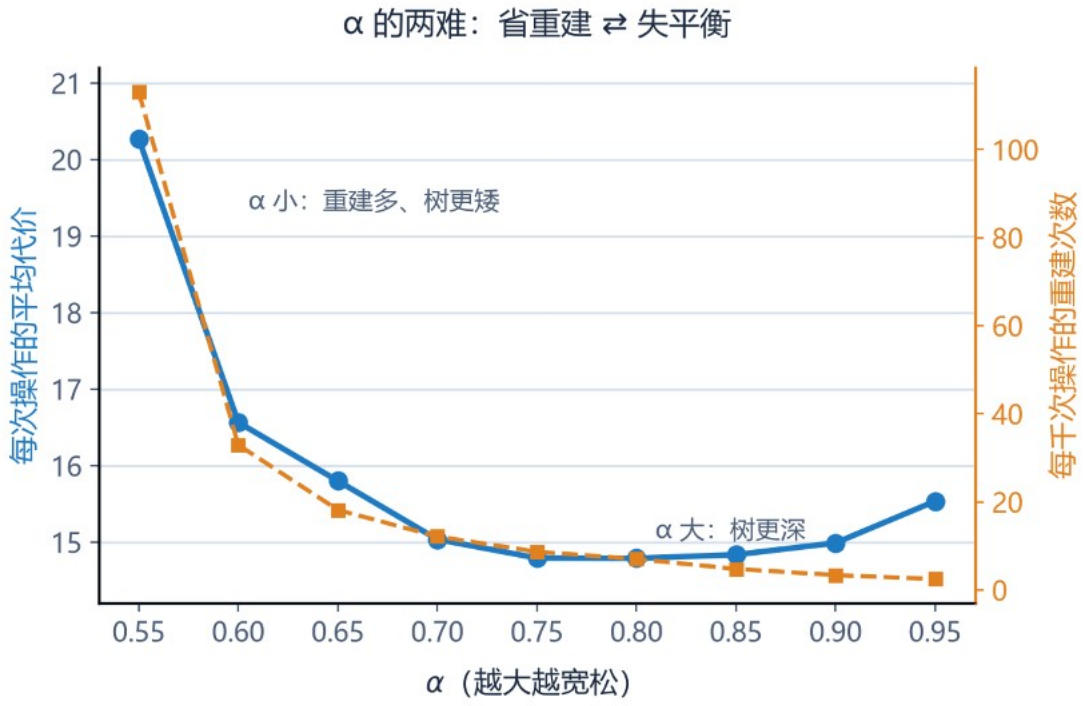
- 读多写少、或结点带「权重」语义时
- 需要严格 1/2- 平衡（子树规模对半）的场合
- 拒绝旋转的复杂实现时

它的「亲戚」

这正是**替罪羊树**（scapegoat tree）的思想：失衡才重建，用势能摊还 —— 区别只在于判据是高度比（替罪羊）还是规模比（本题）。

16-3 实测：平均代价近似 lg n，α 存在最优区间





n 从 10^3 到 10^5，平均代价 12.2 → 19.0，**与 $\log_2 n$ 同步**；
三组 α 曲线形状一致，差别只在常数。

α 太小 → 重建频繁（113 次 / 千次）；α 太大 → 树更深（高 25）。
实测最优区间 α ≈ 0.75–0.85，代价 14.8。

答案

答题清单：五个小问的一句话答案

a

中序取有序序列，再以中位数递归建树： **Θ(m)** 时间、 **O(m)** 空间，原地改写根对象字段，父指针不必动。

b

每下一层，进入的子树规模 ≤ α· 当前，故深度 ≤ **log_(1/α) n = O(lg n)** （ α=0.75 时约 2.41 lg n ）。

c

Δ 是绝对值 ⟹ 每项非负 ⟹ **Φ ≥ 0**； 1/2- 平衡时 |Δ| ≤ 1 ，没有 Δ ≥ 2 的项 ⟹ **Φ = 0** 。

d

失衡 ⟹ Δ(x) > (2α−1)m − 1 ，重建后这部分势清零 ⟹ **c ≥ 1/(2α−1)** 时重建摊还 O(1) 。

e

路径上 O(lg n) 个 Δ 各变 1 （势增 ≤ c·log_(1/α) n ），重建由势能付账 ⟹ **摊还 O(lg n)** ，单次最坏仍 O(n) 。

附 3-1

附：同页的 3-1 —— 多项式只看最高次项

题面

$p(n) = a_d \cdot n^d + \dots + a_1 \cdot n + a_0$，$a_d > 0$，$k$ 为常数。

用渐近记号的定义证明 a–e 五条性质。

关键一步：先证 $p(n) = \Theta(n^d)$

提出主项：$p(n)/n^d = a_d + \Sigma_{i<d}(a_i/a_d) \cdot n^{i-d} \to a_d$（当 $n \to \infty$）。

存在 n_0，$n \ge n_0$ 时 $(a_d/2) \cdot n^d \le p(n) \le (3a_d/2) \cdot n^d$，即 $p(n) = \Theta(n^d)$。

五条性质都是它的推论：o 要求比值 $\to 0$，ω 要求比值 $\to \infty$。

条件	结论	由 $\Theta(n^d)$ 直接推出
$k \ge d$	$p(n) = O(n^k)$	$n^d \le n^k$（$n \ge 1$）
$k \le d$	$p(n) = \Omega(n^k)$	$n^d \ge n^k$
$k = d$	$p(n) = \Theta(n^k)$	同阶，上下界都是常数倍
$k > d$	$p(n) = o(n^k)$	$p(n)/n^k \le (3a_d/2) \cdot n^{d-k} \to 0$
$k < d$	$p(n) = \omega(n^k)$	$p(n)/n^k \ge (a_d/2) \cdot n^{d-k} \to \infty$

要点：不必五条各证一遍。先证 $\Theta(n^d)$，再拿 n^d 与 n^k 比大小；

小 o / 大 ω 只是「比值的极限」写法不同而已。

三个可以带走的结论

① 「最坏一次很贵」不等于「平均很贵」：单次 $O(n)$ 与摊还 $O(\lg n)$ 可以同时成立

② 势函数要选得「恰好被最贵的那次操作吃掉」，Δ 的离差正是这样的量

③ 允许偶尔大修，代码会比旋转平衡树短得多 —— 替罪羊树就是同一个思路

谢谢 · 源码 code/wbt.py · 实验 code/exp.py